An Inductive Logic Programming Approach for ARC-AGI-3

Abu Safwan Md Farhan Zihadul Islam Tasin
K. M. Mehrab Hossain Iftekhar Mohammed Aritro
Department of Computer Science and Engineering
Brac University

***Abstract*—ARC-AGI-3, an interactive benchmark consisting of 135 games that have no explicitly stated win-condition, has become one of the most important benchmarks for measuring frontier AI intelligence in recent times. Even though humans excel at these games, state-of-the-art AI performance remains subpar. Because of the extreme sparseness of reward in these games, Reinforcement Learning approaches, the de facto paradigm for goal-driven interactive tasks, may perform poorly. Under such reward-sparse conditions, we explore Inductive Logic Programming (ILP), a type of Machine Learning, as the alternative. Although ILP has been tested on previous iterations of ARC-AGI, its application to ARC-AGI-3 remains limited. We first show why some Reinforcement Learning systems may fail on ARC-AGI-3 and then show how ILP performs on this benchmark.**

***Keywords*—ARC-AGI-3, Inductive Logic Programming, Reinforcement Learning**

I. Introduction

A. Background

ARC-AGI-3 is the third version of the ARC-AGI series and the hardest version. The first two types, ARC-AGI-1 [10] and ARC-AGI-2 [11], were static puzzle-based benchmarks. There, the agent was asked to understand a pattern by being shown a fixed grid. But ARC-AGI-3 is totally different. This is the first fully interactive reasoning benchmark. Here, the agent is left in a novel game environment, without any instructions, without any rules, and without any stated goal. The agent has to understand how the environment is working by exploring, has to understand the goal, and take action according to that.

Humans can score 100% on this benchmark. But as of June 2026, the most powerful frontier AI systems—including Claude Opus 4.6—scored below 1.5% on the leaderboard snapshot recorded at that date [16]. This gap still exposes a fundamental limitation of current AI.

B. Motivation

To solve these limitations on selected aspects, this research presents ILP as an alternative approach to RL. Unlike RL, which requires multiple repetitions to solve a level with a good score, ILP can generalize an induction purely from a few examples. This quality of ILP makes it more compatible with ARC-AGI-3.

C. Problem Statement

The three main challenges that exist for any agent are outlined in ARC-AGI-3.

1) Goal Discovery: The reward signal is not defined explicitly for the agent. The reward function tells the agent if it is on the right path in the standard RL. The goal must be determined by the agent in ARC-AGI-3. Current RL methods find it very hard to implement this type of unsupervised goal discovery.

2) World Model Induction: The agent needs to create a model of the environment within itself. It must learn that certain behaviors result in certain consequences. This model should be developed based on the agent's personal experiences only. The environments are developed in a way that each one is fundamentally different from another. As a result, all the previously used pattern-matching techniques do not work effectively. To succeed in each level, the agent has to learn every level by experiencing that level.

3) Efficiency: Completing a level is not enough. The agent has to find a solution to it using the minimum number of actions. There is a scoring system that awards efficiency. Humans deal with these environments very effectively and rapidly with very few actions. This is more efficient than what current AI systems are capable of.

D. Objective

Thus, the 4 contributions we make in this paper are:

1) We compare an RL and a random agent on ARC-AGI-3 and show that on ARC-AGI-3, an RL agent is equivalent to a random agent.
2) We propose PermanentLearner, a symbolic RL approach built on top of ILP(RL) [4], which, unlike RL, is able to learn the underlying rules of the game permanently.
3) We test our approach on 3 toy games that embody the spirit of ARC-AGI-3 to some extent and show that PermanentLearner learns the underlying rules of the game in symbolic form with extreme sample efficiency.
4) Finally, we introduce InteractivePopper, which is able to beat ls20 level 1.

E. Scope and Limitations

This paper is strictly dedicated to the ARC-AGI-3 benchmark. It doesn't endeavor to make a universal artificial intelligence system. These controlled, scenario-independent experiments are meant to test how much promise Inductive Logic Programming actually holds as an alternative, not just

to explain why RL agents fail on ARC-AGI-3. This research makes no claim to a full ILP-based solution covering the whole ARC-AGI-3 environment. Instead it pulls apart two core sub-problems, world model induction and goal discovery, and tests each on its own, since solving the full scenario problem just wasn't realistic given the timeline and scope of this study.

II. Literature Review

A. Preliminaries

1) ARC-AGI-3:

Structure of the Games: There are several game environments in the benchmark. There are at least 6 levels in each of the games. New mechanics are added to each level, which the agent will need to learn and understand.

Each game environment includes the following:

- **Observations:** A 64×64 grid that has 16 colors available. This is what the agent will see on each turn.
- **Actions:** The agent can have 7 possible actions. They are: `ACTION1` `ACTION2` `ACTION3` `ACTION4` (directional action: up, down, left, right), `ACTION5` (general interaction: select, rotate, execute etc.), `ACTION6` (click action, x, y coordinates: 0–63), and `RESET` (restart current level).

A standard API provides the agent with access to the game. It is given an observation frame, chooses an action and is told if the state changed at each step. Rewards are only given once a level has been successfully completed. This means that the reward signal is very sparse.

There are 135 environments in total for the benchmark. These include 25 that are publicly available for development. Of the rest, 55 are semi-private, used for the leaderboard evaluation. The rest of the 55 are completely private and are dedicated for the final competition.

Scoring: RHAE: The Relative Human Action Efficiency (RHAE) is the scoring method used in ARC-AGI-3 [15]. Not only does it check if the agent reaches a level, but also whether it does it efficiently or not, in relation to a baseline of human performance. $\left(\frac{\text{Human actions}}{\text{Agent actions}}\right)^2$ is the per-level score. If a human took 10 actions to complete a level, and the agent took 100 actions, the agent gets 1% for the level.

2) Reinforcement Learning: Reinforcement Learning (RL) is a type of machine learning. Here an agent learns by interacting with an environment. At each step, the agent observes the current state, takes an action and receives a reward. The agent's task is to discover a policy. It maximizes total reward over the course of time.

RL is mathematically formulated as Markov Decision Process (MDP) [1]. MDPs have five components:

- S: All possible states.
- A: All possible actions.
- Reward Function $R(s, a)$: This gives the reward when action a is taken by an agent in state s.
- Transition Function $P(s' \mid s, a)$: The probability of being in state s' given that the action taken in state s is a.
- γ: The discount factor, where $\gamma \in [0, 1)$. It entails the value of the agent for the future reward vs the immediately available reward.

The agent learns a policy $\pi : S \to A$ that maximizes the expected return:

$$G_t = \mathrm{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \right] \tag{1}$$

Here, G_t is the expected return from time step t, R_{t+k+1} is the reward received at a future time step $t + k + 1$, and γ is the discount factor that controls how much the agent values future rewards relative to immediate ones.

3) Inductive Logic Programming: Inductive Logic Programming (ILP) is a subfield of machine learning. ILP learns hypotheses in the form of logic programs instead of numerical models such as weight matrices or decision boundaries [2], [9].

There are three parts to a formal ILP task (B, E^+, E^-), where:

- B is the background knowledge: contains the facts and relationships that are already known.
- E^+ is a set of positive examples: contains the facts that the learned hypothesis should explain.
- E^- is a set of negative examples: contains the facts that the learned hypothesis should not explain.

Given these inputs, the goal of an ILP system is to find a hypothesis H, which is a set of logical rules. The hypothesis should explain as many positive examples in E^+ as possible while avoiding the negative examples in E^-. The rules in H are usually written in the following form:

$$\text{head} \leftarrow \text{body}_1, \text{body}_2, \ldots, \text{body}_n \tag{2}$$

This means that the head is true whenever all the atoms in the body are true. Since B, E^+, E^-, and H use the same logical language, the rules learned by an ILP system can also be added to the background knowledge. This allows the system to reuse knowledge learned from one task in other tasks [9].

The set of all possible hypotheses that can be created from the given background knowledge is called the hypothesis space. In ILP, learning is treated as a search through this space. The system looks for a hypothesis that is logically consistent with the given positive and negative examples.

Several properties of ILP make it relevant. First, ILP can learn from a small amount of data compared to many statistical machine learning methods. This is useful when an agent has only a limited number of interactions with an environment. Second, ILP produces logical rules that are easier to understand and interpret than the parameters of neural networks. Third, ILP naturally works with relational representations, that allow it to describe relationships between different entities. These relationships can be difficult to represent using fixed-size feature vectors.

Popper: Popper is a modern ILP system based on the learning-from-failures approach [7]. Instead of searching through the entire hypothesis space directly, Popper first generates a possible hypothesis and tests it against the examples. When the hypothesis fails, however, fails to cover enough positive cases, or to cover a negative case, or to include unnecessary rules, Popper uses that information to add constraints.

These restrictions reduce the search space that is not likely to be a solution.

This process is repeated in a generate-test-constrain loop. Popper uses Answer Set Programming (ASP) to implement this process, that helps it search efficiently through a large number of possible logic programs.

B. Review of Existing Research

1) Symbolic Reinforcement Learning using Inductive Logic Programming: Kunii (2018) proposed ILP(RL), a proof-of-concept that pairs ILP with RL to learn state transitions in grid maze environments [4]. Rather than learning a value function over states and actions, the agent works out a hypothesis describing how the environment moves from one state to the next, and an ASP planner then uses that hypothesis to string together a sequence of actions toward a terminal state.

ILASP2i does the induction work. At each step the agent watches a state transition happen and turns it into a context-dependent positive example, following the CDPI format from Section II-A3. If the current hypothesis can't account for a new example, ILASP gets called again to patch it up. Once the agent finds a terminal state and has a hypothesis good enough to work with, Clingo solves an ASP program to produce an action plan.

Testing happened in simple VGDL-based grid mazes across 30 independent runs of 100 episodes each, 250 steps per episode. ILP(RL) hit the optimal policy by around episode 40 on average; Q-learning took until roughly episode 60. Each ILASP call took 5.58 seconds on average in the baseline environment, with about 12.83 calls needed per episode. Adding a teleport link as a new transition concept pushed the search space from 1690 candidate rules up to 32755, and average induction time jumped to 95.47 seconds. In the transfer experiments, a hypothesis learned in one maze carried over cleanly to a structurally similar one, letting the agent plan from episode 0 with no re-induction needed.

Limitations.: Everything here was tested in small, deterministic, discrete grid mazes only. Induction was the main bottleneck, and it slowed the whole approach down even on simple problems. The framework also needed a hand-written language bias before it could learn anything, assumed the agent could see adjacent cells, and couldn't run ASP planning until a terminal state had actually been found — which cuts against goal-directed MDPs where rewards are sparse.

2) The ILASP System for Inductive Learning of Answer Set Programs: Law et al. (2020) walk through the ILASP system in detail, covering its learning framework, the kinds of examples it accepts, and how its algorithms changed across four versions [6]. ILASP learns Answer Set Programs instead of Prolog programs, which opens up a lot more ground to work with. Most ILP systems only learn definite rules, but ILASP can handle normal rules, choice rules, and both hard and weak constraints, so it stretches to problems like preference learning, common-sense reasoning with defaults and exceptions, and theories that aren't strictly deterministic.

An ILASP task has three parts: background knowledge B, a mode bias M that sets the rule space S_M, and a set of examples E. The task is to find a hypothesis $H \subseteq S_M$ where $B \cup H$ covers every example in E, preferring the shortest H that does the job (Occam's Razor, essentially). ILASP works with three kinds of examples. Positive and negative examples describe partial interpretations that either should or shouldn't come out as answer sets of the learned program. Context-dependent examples go further by giving each example its own local background knowledge, a context C, so the property gets checked against $B \cup H \cup C$ instead of just $B \cup H$. Ordering examples, which come into play for learning weak constraints, express a preference between pairs of answer sets rather than saying whether something exists at all.

Limitations.: As documented by Law et al., all four ILASP versions construct the full rule space before learning starts, and that construction cost is the core scalability bottleneck [6]. The paper points to FastLAS [5], a related system built alongside ILASP, which sidesteps this by computing only an OPT-sufficient slice of the rule space. Though for now it only works on non-recursive, observational predicate tasks. The wider gap between what ILASP can express and how well it scales to large, real-world rule spaces is still an open problem.

3) Duck Harness: Duck Harness is an open-source agent harness for ARC-AGI-3 built by Tufa Labs. It landed among the top entries in the ARC-AGI-3 Kaggle competition [17].

Duck Harness fits a wider pattern showing up across the strongest current ARC-AGI-3 approaches: treat the benchmark as a code-generation and tool-use task for an LLM, rather than as a classical sequential decision-making problem for model-free reinforcement learning to grind through. This framing gets around the reward-sparsity problem that motivates the ILP-based approach in this thesis, but it does so by leaning on the world knowledge and general reasoning already baked into a large pre-trained model, not on any explicit, inspectable model of environment dynamics learned from interaction.

4) Deep Q-Networks: Deep Q-Networks (DQN) were introduced by Mnih et al. [3]. It is the first deep learning model that learned control policies directly from raw-pixel inputs via RL without any handcrafted features or feature extraction. Previous RL techniques involved manually designed features, and applied them to visual input. However, when using more complex models, Q-learning would be unstable and sometimes may not converge during learning. In contrast, DQN did this using Q-learning and a convolutional neural network, which was trained through stochastic gradient descent.

Experience replay is used in DQN for stability in training. The agent learns from all of these experiences and will randomly draw from its memory for training the network. This will keep the training stable and not have two experiences be similar to each other. In training, actions are taken according to an ϵ-greedy policy (where ϵ is decreasing linearly over time) to ensure a balance between exploration and exploitation.

5) Summary of Key Findings: Distinct from all the other agents in Table I, PermanentLearner was examined using a

TABLE I
COMPARISON OF ARC-AGI-3 PREVIEW AGENT COMPETITION APPROACHES.

Approach	Observation handling	Learning strategy	Main limitation
StochasticGoose Score: 12.58% Levels: 18 [14]	CNN over 64×64 frames	RL with replay and action prediction	Semi-random exploration; assumptions about full state observability
Blind Squirrel Score: 6.71% Levels: 13 [12]	Frame segmentation	RL	Segmentation alone insufficient for private environments
Explore It Till You Solve It Score: — Levels: 30/52 [13]	Visual decomposition	Training-free graph exploration	State-space growth makes it intractable in large environments
PermanentLearner Score: this thesis	Structured symbolic states	Popper + ASP planning	No perception layer; small controlled environments only

controlled environment rather than on ARC-AGI-3. It is added here solely for the purpose of showing the contrasts of learning strategies against the competition entries.

III. PROPOSED METHODOLOGY

A. RL vs. Random Agent Methodology

We test on only the first level of `ls20`, one of the 25 public games of ARC-AGI-3 [15]. We use a simple reward scheme: a +1 reward for when the agent experiences the win-condition, and 0 reward for all actions other than that. We use a DQN [3] agent from Stable-Baselines3 [8] with a custom CNN. See Table II for hyperparameter values.

B. PermanentLearner Methodology

The interaction of the environment, first stated in terms of state transitions and then followed by inducing hypotheses based on those transitions and then using the hypotheses in an ASP planner to generate action sequences, and then repeating the cycle of interactions with the environment is the same in PermanentLearner as in ILP(RL) [4]. The name is based on this central concept: In standard RL the "learning" from one level isn't transferred to be stored in generalizable knowledge, but rather requires a lot of repetition each time (as shown in Contribution 1). In contrast, the induced rules in PermanentLearner are explicitly represented in symbolic form, and upon successful learning do not need to be learned again. Its greatest attribute is its persistence.

In the original ILP(RL) system [4], they had used ILASP2i [6] as the induction server. This system was not able to be run reliably in our experimental environment, so ILASP was replaced by Popper [7] in PermanentLearner, with the rest of the pipeline (ASP planner and evaluation methodology) remaining the same to facilitate a fair comparison with the original design. To make the replacement possible, three important changes were made to the examples of the ILASP: (a) Each example had to be translated into a flat example, stating whether the goal is achieved or not; (b) The action in each example had to be a relation with a direction, e.g. `step_up`, `step_down`, `step_left`, `step_right`, which means that the action and the direction of the movement are not conflicting; and (c) The examples contained an "open" cell, but this one had to be represented as a positive predicate instead of stating "not wall", because Popper's bias is not the same as ILASP's.

PermanentLearner's key learning process is an iterative one: the next experience is captured, and the existing hypothesis is tested to see if it can explain the experience, only invoking Popper if the hypothesis doesn't explain the experience. This same general approach without restricting it to any particular domain, is used for both transition rule induction and goal induction (see Section III-C). Q-learning is maintained as the baseline from the original model to isolate the effect of changing the induction backend only, and observe it. The main engine is based on Learning from Failures (LFF) framework, and is a version of Popper v3.1.0, committed at `227c30f` [7].

C. PermanentLearner Experimental Setup on 3 Toy Examples

We wanted to test if PermanentLearner's induction method can handle this task. We designed three small symbolic toy games. We deliberately set aside ARC-AGI-3's unsolved perception problems. These games have no explicit goal concept in a controlled setting.

The games gradually grow complex according to their target predicate

Unary game: the goal depends on a single object's property for example: an object of a specific color must be there.

Relational game: the goal depends on a relationship between two objects for example: two similar objects will share the same attributes.

Compositional game: the goal combines multiple conditions together.

The agent is never told about the goal predicate, it only sees whether a state is a success or a failure. The agent uses the coverage check then reinducts the loop from section III-B. By doing this it builds a hypothesis about the hidden goal predicates from its observations.

We have tested the learned rules later in a held out set of unseen states. This experiment ensures that the agent is not memorizing the rules rather it's actually generalizing them. We built these held out states using the same method but we kept them out of the induction phase.

We gave the learned goal predicate directly to the ASP planner. This step proved that goal discovery is not just a classification exercise; it showed that the agent can actually

TABLE II
HYPERPARAMETER SPECIFICATIONS AND NETWORK ARCHITECTURE FOR DQN ON ARC-AGI-3

Category	Hyperparameter	Value / Specification
Environment & Input	Observation Space Dimension	$1 \times 64 \times 64$
	Palette Value Range	$\{0, 1, \ldots, 15\}$ (Categorical)
	Normalization	Disabled (`normalize_images=False`)
	One-Hot Indicator Channels	16
Feature Extractor (CNN)	Convolutional Layers	3 layers ($16 \to 32 \to 64 \to 64$)
	Kernel Size, Stride, Padding	3×3, Stride 2, Padding 1
	Spatial Pooling	Adaptive Average Pooling to 4×4
	Flattened Feature Size	1,024 ($64 \times 4 \times 4$)
	Latent Projection (d)	256 units (`Linear` + `ReLU`)
Q-Network Head	Architecture	2-layer MLP
	Hidden Units	$[64, 64]$
	Hidden Activation	ReLU
	Output Layer	Linear ($64 \to \lvert\mathcal{A}\rvert$)
DQN Algorithm & Training	Optimizer	Adam
	Learning Rate (α)	5×10^{-4}
	Discount Factor (γ)	0.99
	Replay Buffer Capacity (N)	30,000
	Batch Size (B)	64
	Warmup / Learning Starts (T_{start})	1,000 steps
	Update Frequency	1 gradient step per 4 env steps
	Target Network Update Interval (C)	250 steps

plan and reach its own goal using what it learned. With this we completed the loop from raw interaction to planned action.

Each game was run on multiple independent random seeds to confirm consistent results. Our main sample-efficiency metric was the number of examples needed for the hypothesis to stabilize.

D. InteractivePopper

Finally, we were able to build an agent that interacts with a real ARC3 game: it played and completed ls20 level 1 within 59 actions. We call it InteractivePopper. InteractivePopper starts with some background knowledge and some predefined assumptions.

1) Predefined Assumptions and Background Knowledge: Predefined Assumptions:

1) It uses connected-component labeling for vision. So, it assumes that adjacent tiles of same color constitute a single object.
2) It can model the 64x64 grid using only 2 square channels at a time.

Background Knowledge and Bias: InteractivePopper uses some pre-given hints as background knowledge. For example, the concept of an opposite point is given to InteractivePopper as a fact. It uses this concept of oppositeness later to induce the rotation rule in ls20.

Moreover, the bias file specifies that rules can have at most 3 conditions and 4 variables, so that the search for Popper is easier.

2) InteractivePopper Workflow:

1) At time step t, it receives only three things from the environment: the 64x64 grid, the current level of the game from the ARC game, and the current actions it has available. It receives no explicit goal.
2) It learns what its actions do by trying them out one by one. For example, it stores its current coordinate, presses ACTION1 (say right), and then receives its next coordinate in $t + 1$. Then, it subtracts the initial coordinate from the final to figure out that ACTION1 corresponds to "RIGHT".
3) The agent compares consecutive frames, and detects the region where there are pixel changes. Then a bounding box of size NxN is snapped to that region. It records which pixels were occupied and which were not before the move.
4) Now, if the goal is known, the agent just uses BFS. But if the goal is not known, the agent uses curiosity to select its action. It prioritizes states which are unvisited.
5) The chosen action is sent to the environment, and executed.
6) Observe outcome.
7) InteractivePopper generates positive and negative examples on the fly, where active pixels are labelled positive, and blank pixels negative.
8) When predictions don't match with the real state of the environment, Popper is invoked. Popper induces the rule and then feeds it back to step 4. Go to step 1.

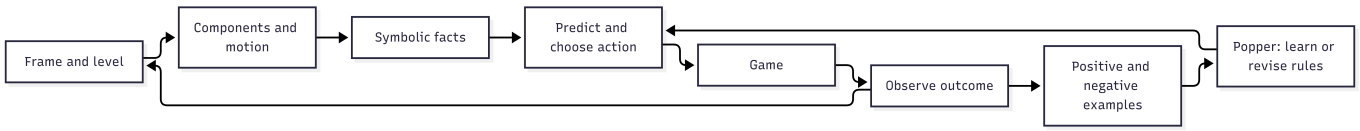


Figure 1. InteractivePopper Workflow

IV. Result Analysis

A. RL vs. Random Agent Results

Our first experiment focused on the behavior of the RL agent on ARC-AGI-3. The per-seed results are shown in Table III, the verified recordings in Table IV, and the learning curve in Figure 2. The RL agent performed the same as a random agent when it was given to complete a game level within a limited number of steps. Though its performance got better when it was given the chance to repeatedly play the same level. It is notable, as this repeated exposure gives the agent a chance to remember the earlier actions and use them later in the gameplay. So this minimal improvement should not be considered as a quick adaptation to a new game environment. Our original thesis also focuses on this memorization behaviour rather than quick adaptation.

B. PermanentLearner Initial Results

Secondly we have used Popper to learn the transition rules of a symbolic maze environment. Our goal for this experiment was to determine if the agent is able to learn a general world model from observed state transitions, and later use that model for planning.

In an initial smoke test, the agent collected 28 transitions from the `aaa_small` maze, and Popper learned an eight-rule transition model that produced 28 true positives, 112 true negatives, 0 false negatives, and 0 false positives against held-out transitions. This model let the planner reach the goal in 7 steps.

We then evaluated Popper against a Q-learning baseline over 30 independent seeds, using 100 episodes of at most 250 steps each. Table V reports the results.

The convergence metrics are not identical. PermanentLearner's value records the last episode that triggered re-induction, whereas Q-learning's value records the first evaluation matching its final policy. The per-episode traces are therefore the more direct sample-efficiency evidence.

The final hidden goal experiment combined goal learning with planning across three symbolic games — unary, relational, and compositional — each evaluated over 10 seeds. Table VI reports the results.

Because the generators are small and structured, these perfect scores demonstrate pipeline feasibility rather than performance on the full ARC-AGI-3 problem.

This hidden goal experiment also tells us that the agent did not require a large number of examples in these small environments. The average point of convergence was between 2.5 and 5.6 examples, depending on the types of the goals. This is consistent with the motivation for using inductive logic programming in these small, structured environments; it does not establish general sample-efficiency or scalability. A hypothesis can be learned from a relatively small number of structure examples. Our thesis also describes ILP as a method that can learn logical rules from limited data.

The transfer experiment adds one more perspective on symbolic knowledge reuse. A transition model learned on `aaa_small` was carried over to the `experiment3_after` maze, and across five seeds the agent achieved 100% success with a 21-step plan every time. This shows the learned transition representation could support planning in a different registered maze environment. The original target, `aaa_medium`, was unavailable due to a missing level file, so `experiment3_after` served as the closest available alternative. This result is therefore better read as a limited transfer experiment than a full evaluation of general transfer ability.

C. InteractivePopper Results

For our fourth experiment we tested InteractivePopper, a combination of pixel/movement rules generated by Popper, a hand-crafted empirical table of action displacements, and breadth first search on the learned model. This agent does not employ an ASP planner, and the system as a whole should not be considered as being entirely ILP based. Tested on real public ARC-AGI-3 games (`ls20`, `ft09`, `cn04`) and the game files were downloaded using the official ARC Prize download toolkit, but the games were not played as a live scorecard session (Table VII).

On `ls20`, the 1st level is finished on both runs (59 actions and 60 actions, respectively). If we base it on the human level (Table IV) it comes to an RHAE of about 13.9% and 13.4% on the two seeds, respectively, which is a genuine, if not substantial, achievement. These two runs then went on to Level 2 and applied the rules they had just learnt in this level without re-induction, but the agent failed to select an action in this level: in the controlled synthetic corridor experiment (Section III-C), the same planning branch had succeeded. The agent could not finish any level of the other games (here `ft09` and `cn04`) within the budget of the action (Table VII).

The percentage of regions that change (Table VIII) is also significantly lower than the overall percentage, because many of the registered regions didn't change from step to step, which introduces a bias into the overall statistics. These results are not an evaluation performed blindly, without trying to take into account what the developers were looking for, but the results of a system which was being iteratively modified as it progressed, in response to observing `ls20` and the other

TABLE III
COMPARATIVE PERFORMANCE ACROSS 5 INDEPENDENT SEEDS ON ARC-AGI `ls20` (LEVEL 1, 60,000 ACTIONS)

Run / Seed	Random Agent Baseline		DQN Agent (Linear ϵ-Decay)		
	1st Win Step	Total Wins	1st Win Step	ϵ at 1st Win	Total Wins
Seed 1	53,596	1	15,798	0.53	479
Seed 2	14,150	1	24,755	0.26	6
Seed 3	55,442	1	18,528	0.44	83
Seed 4	47,254	2	8,772	0.74	311
Seed 5	27,073	1	13,797	0.59	167
Mean	**39,503.0**	**1.20**	**16,330.0**	**0.51**	**209.20**
Std. Dev.	±18,097.0	±0.45	±5,908.4	±0.18	±188.50
Min	14,150	1	8,772	0.26	6
Max	55,442	2	24,755	0.74	479

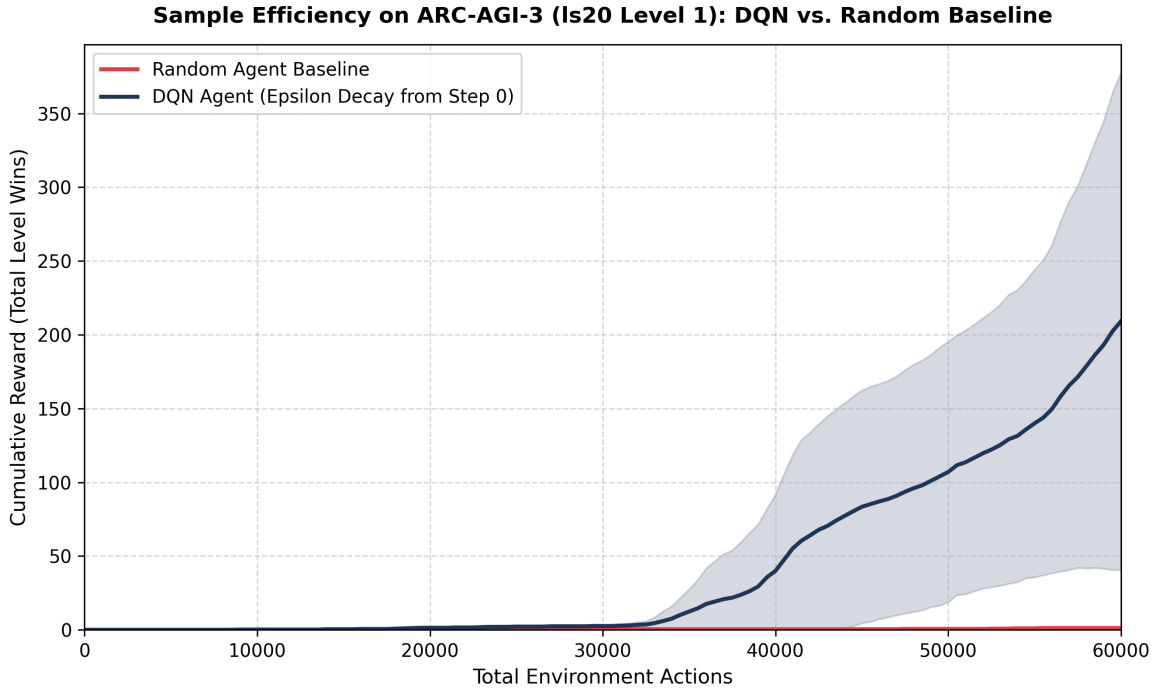


Figure 2. Sample efficiency on ARC-AGI-3 (`ls20` Level 1): Cumulative level completions over 60,000 environment interactions comparing the Random Agent Baseline against the DQN Agent across 5 independent random seeds. The solid lines represent empirical means, and the shaded regions indicate ±1 standard deviation.

TABLE IV
VERIFIED ARC-AGI ONLINE RECORDINGS ON `ls20` (HUMAN BASELINE $H = 22$ ACTIONS)

Recording	Milestones Unlocked	Actions	RHAE	Public ARC Scorecard
1	Milestone 1: First Completion	132	2.78%	`69084f04-34a3...`
2	Milestones 2–4 (25%, 50%, 75%)	23	91.49%	`e3ac1ce9-2736...`
3	Milestone 5: Achieve 100% RHAE	**15**	**115.00%**	`3f9516ff-8d3f...`

games that were public. No ablation or RL comparison was made in order to isolate the specific contribution of Popper from that of the hand built displacement table.

D. Statistical Analysis

The analysis here is descriptive.

For the baseline comparison between Popper and Q-learning, each method ran on 30 independent seeds. Results

TABLE V
PERMANENTLEARNER VS. Q-LEARNING BASELINE ON AAA_SMALL (30 SEEDS)

Evaluation Metric	PermanentLearner	Q-learning
Task Performance (Deterministic Maze)		
Success Rate	1.00 ± 0.00	1.00 ± 0.00
Mean Episode Reward	4.00 ± 0.00	4.00 ± 0.00
Mean Solution Steps	7.00 ± 0.00	7.00 ± 0.00
Learning Dynamics & Computational Cost		
Convergence (Episodes)*	$\mathbf{1.27 \pm 0.51}$	6.93 ± 1.12
Induction Calls	4.60 ± 0.71	—
Induction Time (s)	6.23 ± 1.32	—

* Convergence is defined per backend: for PermanentLearner, episodes until hypothesis convergence; for Q-learning, episodes until the evaluation policy matches the optimal 7-step path.

TABLE VI
HIDDEN-GOAL INDUCTION AND PLANNING RESULTS, 10 SEEDS PER GAME

Game	Ex. to conv.	Held-out	Precision	Recall	Plan succ.	Plan len.
Unary	2.500±0.500	1.000	1.000	1.000	1.000	3
Relational	5.600±4.964	1.000	1.000	1.000	1.000	6
Compositional	5.600±4.176	1.000	1.000	1.000	1.000	4

TABLE VII
INTERACTIVEPOPPER PERFORMANCE IN PUBLIC ARC-AGI-3 GAMES

Game	Action of 1st Level Clear	Total Actions	Levels Completed	Outcome
ls20 (seed 0)	59	126	1	Game over, Level 2
ls20 (seed 1)	60	127	1	Game over, Level 2
ft09	—	80	0	Budget exhausted
cn04	—	60	0	Budget exhausted

TABLE VIII
INTERACTIVEPOPPER PREDICTION ACCURACY

Game	Position Correct	Bitmap Correct	Correct on Changing Regions
ls20 (seed 0)	113/119	166/168	12/14
ls20 (seed 1)	115/120	162/164	6/8
ft09	—	110/113	1/2
cn04	—	0/1	0/1

are reported as mean and population standard deviation across success rate, mean reward, mean number of steps, induction time, number of induction calls, and convergence episode.

The independent goal-learning experiments used 10 seeds per goal template, tracking the number of examples needed to converge and agreement on held-out states. Experiments were also run across three game types using 10 random seeds each, measuring learning speed, accuracy, and planning success. Formal statistical tests such as t-tests or p-values were not used.

E. Discussions

This result also shows us the cost of the symbolic approach; Popper requires time to search for a consistent hypothesis and the transition experiment requires several induction calls in the larger evaluation. The agent needs fewer trial-and-error interactions to understand its environment compared to standard RL. The tradeoff is computational speed, as the environment adds more objects and actions, and the number of possible rules expands rapidly. Searching through these rules requires heavy computation making it the slowest part of ILP.

Several limitations persist in the current results. Firstly the transition and the hidden goal experiments use small and structured environments. The symbolic states are already in logical form so the experiments do not solve the complete perception problems of ARC-AGI-3.

Secondly, the Popper implementation follows the original collect/check/re-induce pattern used by ILASP2i. But its

context-reified task and compiled bias are not a byte-for-byte reproduction of ILASP2i's search space. The results show that Popper works end-to-end as a replacement induction backend, though not as an exact one-to-one substitution.

For our third limitation, the hidden goal experiments were performed on synthetic symbolic environments. The 100% results show successful learning for the tested generators but it is not guaranteed that the same level of accuracy can be achieved on much larger and more complex ARC-AGI-3 game levels.

Fourth, the transfer result is limited because the intended `aaa_medium` registration could not be used, so we chose a different maze, `experiment3_after`. As there were no cross-maze Q-learning transfer comparisons, there are no conclusions on comparative transfer performance from the experiments.

Fifth, the current Popper backend is focused on the ordinary four-action transition setting. So the updated transition does not claim support for more complex transition types like links or teleportation. These cases require additional rules and further testing.

Lastly, the statistical analysis is descriptive. The experiments show very stable results in the tested settings, but we need to do larger experiments on more complex environments to pass a claim about efficiency, scalability, and generalization.

The results support the main idea of the new approach, i.e., symbolic learning can be used to separate world-model induction from goal discovery and then connect the learned knowledge to planning. Our current experiment proves that our idea is effective in a controlled environment, and the rules that are learned can be very accurate, interpretable, and usable for action planning. But these results should be treated as a proof of concept rather than taking it as evidence that the complete ARC-AGI-3 problem has been solved.

V. Conclusion

Overall, the results demonstrate that the ILP-based method can learn world models and find goals with fewer examples than regular reinforcement learning in controlled and scenario-independent environments. It's also easier to learn since the acquired rules can easily be scrutinized. However, these results are to be considered proof of concept, rather than the full answer. This approach has not yet been used to solve the entire set of problems in ARC-AGI-3, which are presented in the form of scenarios.

A. Recommendations for Future Work

The main work for the future will be to resolve the perception problem. The current method of this paper cannot be executed on ARC-AGI-3's [15] raw 64x64 grid due to its limitations.

Secondly, the current Popper method's bounded induction loop should carefully follow the ILASP-based online iterative loop; that's why it should extend to be a true online version. Thirdly, both world model induction and the goal discovery method should be evaluated in more complex, diverse, and large environments to verify the scalability, particularly including complex or coordinate-based action and teleportation or link mechanics to extend the transition backend. Fourthly, to enlarge the domain of this empirical investigation, depending on the descriptive statistics is not enough to get reliable data; rather, formal statistical tests should be done.

Therefore, if the perception problem becomes solvable, goal discovery and transition learning can be incorporated into a unified agentic loop; this approach can create a path to solve the combination of all three challenges, i.e., goal discovery, world model induction, and efficiency together.

Finally, InteractivePopper was not tuned for all the games, so future work needs to test it on those unseen environments. Then Popper needs to be separated from the hand-built displacement table to isolate its contribution, and an investigation should be done to find out why it worked on the controlled test (Section III-C) but could not take any action at the game's Level 2.

Acknowledgement

We would like to thank our co-supervisor Utsha Kumar Roy for the incredible kindness he showed us throughout our thesis journey. His correspondence helped us organize our thoughts, and finally led us to our final thesis.

We would also like to thank Md Humaion Kabir Mehedi for his kindness, for being our mentor for this journey, and for providing us with all sorts of logistical help.

Most importantly, we would like to thank our supervisor Dr. Chowdhury Mofizur Rahman for treating us with extreme patience and kindness and for giving us immense intellectual freedom. Our regular meetings kept us on our toes and helped us finish our work on time.

Finally, we would like to thank all of the members of faculty for providing us with knowledge.

Bibliography

[1] R. S. Sutton, "Learning to predict by the methods of temporal differences," Machine Learning, vol. 3, no. 1, pp. 9–44, 1988. doi: 10.1007/BF00115009

[2] S. Muggleton, "Inductive logic programming," New Generation Computing, vol. 8, no. 4, pp. 295–318, 1991. doi: 10.1007/BF03037089

[3] V. Mnih et al., "Human-level control through deep reinforcement learning," Nature, vol. 518, no. 7540, pp. 529–533, 2015. doi: 10.1038/nature14236

[4] K. Kunii, "Symbolic reinforcement learning using inductive logic programming," M.S. thesis, Imperial College London, Sep. 2018. [Online]. Available: https://github.com/921kiyo/symbolic-rl/blob/master/report.pdf

[5] M. Law, A. Russo, E. Bertino, K. Broda, and J. Lobo, "Fastlas: Scalable inductive logic programming incorporating domain-specific optimisation criteria," in Proceedings of the AAAI Conference on Artificial Intelligence, vol. 34, 2020, pp. 2877–2885. doi: 10.1609/aaai.v34i03.5678

[6] M. Law, A. Russo, and K. Broda, "The ILASP system for inductive learning of answer set programs," in Proceedings of the 24th European Conference on Artificial Intelligence (ECAI), 2020. doi: 10.48550/arXiv.2005.00904

[7] A. Cropper and R. Morel, "Learning programs by learning from failures," Machine Learning, vol. 110, no. 4, pp. 801–856, 2021. doi: 10.1007/s10994-020-05934-z

[8] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann, "Stable-baselines3: Reliable reinforcement learning implementations," Journal of Machine Learning Research, vol. 22, no. 268, pp. 1–8, 2021. [Online]. Available: http://jmlr.org/papers/v22/20-1364.html

[9] A. Cropper and S. Dumančić, "Inductive logic programming at 30: A new introduction," Journal of Artificial Intelligence Research, vol. 74, pp. 765–850, 2022. doi: 10.1613/jair.1.13507
[10] ARC-AGI-1, 2024. doi: 10.48550/arXiv.1911.01547 arXiv: 2412.04604 [cs.AI].
[11] F. Chollet, M. Knoop, G. Kamradt, B. Landers, and H. Pinkard, "Arc-agi-2: A new challenge for frontier ai reasoning systems," arXiv preprint arXiv:2505.11831, 2025. doi: 10.48550/arXiv.2505.11831
[12] W. Dick, ARC-AGI-3 agents: Blind squirrel, GitHub, Accessed: 2026-06-08, 2025. [Online]. Available: https://github.com/wd13ca/ARC-AGI-3-Agents
[13] E. Rudakov, J. Shock, and B. U. Cowley, Graph-based exploration for ARC-AGI-3 interactive reasoning tasks, 2025. doi: 10.48550/arXiv.2512.24156 arXiv: 2512.24156 [cs.AI].
[14] D. Smit, 1st place in the ARC-AGI-3 agent preview competition, Medium, Accessed: 2026-06-08, 2025. [Online]. Available: https://medium.com/@dries.epos/1st-place-in-the-arc-agi-3-agent-preview-competition-49263f6287db
[15] ARC Prize Foundation. "ARC-AGI-3: A new challenge for frontier agentic intelligence," ARC Prize Foundation, Accessed: Sep. 25, 2026. [Online]. Available: https://arxiv.org/abs/2603.24621
[16] ARC prize leaderboard, https://arcprize.org/leaderboard, 2026.
[17] H. Bessis, J. Cottaar, I. Pressman, A. Smit, M. Tešnar, and S. Viel, Duck harness: Winning solution for arc-agi-3 milestone 1, Tufa Labs Research Blog, Jul. 2026. [Online]. Available: https://tufalabs.ai/research/duck-harness/